SIMATS ENGINEERING

Department of Information Technology

DSA01 – Object-Oriented Programming with C++

ASSIGNMENT 1 – CO1

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs.

Student Name	U. Ananya	**Register Number**	192424382
Department / Section	AI&DS	**Academic Year / Semester**	3rd year
Faculty Name	AMIRTHA S	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	06-11-26	**Date of Evaluation**	
GitHub Repository	https://github.com/Ananya764260/DSA0114-192424382-C-.git	**Compiler**	g++ 13.3.0, -std=c++17 -Wall -Wextra

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Object-Oriented Analysis	4		
2	Tokens, Keywords, Identifiers and Constants	3		
3	Data Types and Compatibility	4		
4	Implicit Conversions	4		
5	Operators and Precedence	4		
6	Control Structures	4		
7	Procedural vs Object-Oriented	4		
8	Input and Output	4		
9	Implementation and Testing	4		

10	Expectation vs Execution	4		
11	Final Recommendation	4		
12A	Re-evaluate: Impact Analysis	2		
12B	Re-evaluate: Evaluation of Original Design	2		
12C	Re-evaluate: Adaptation or Redesign	2		
12D	Re-evaluate: Evidence-Based Defence	2		
	TOTAL	50		Percentage:

OVERALL REMARKS

Student Signature	**Faculty Signature**	**Program Director**

ASSIGNMENT 1 – CO1

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

Scenario: Namma Metro Smart-Card Fare System

A city metro rail divides its network into three zones: Inner, Middle and Outer. The basic fares are Rs 10, Rs 20 and Rs 30 respectively. A 20% surcharge is applied during peak hours (8–10 AM and 5–8 PM). Passengers use stored-value smart cards, and a journey is allowed only if at least Rs 50 remains after deducting the fare. Students and senior citizens receive concessions.

The system must avoid floating-point errors, incorrect peak-hour calculations, negative card balances and duplicated fare values.

Assumptions used in this answer:

- Students and senior citizens receive a 50% concession on the base fare.
- Peak surcharge is applied after the concession.
- Money is stored in integer paise.
- Peak hours are 08:00–09:59 and 17:00–19:59.
- At least Rs 50 must remain after every paid journey.

TASK 1 – Understand the Problem and Identify the Objects

1. Object-Oriented Analysis of the Metro Fare System

The metro fare system contains several objects that perform different responsibilities. Object-oriented programming helps organize the system by combining data and behaviour into classes.

1.1 Objects, attributes and behaviours

Object	Attributes (Data)	Behaviours (Methods)
SmartCard	Card ID, balance	balance(), deduct(), recharge(), canTravel()
Passenger	Name, passenger category	isConcessionEligible()
Journey	Zone, hour, minute	isPeak(), isValid()
Zone	Inner, Middle, Outer	Represents travel zones
FareTable	Base fares, surcharge, concession	calculateFare()

SmartCard: Stores the passenger's balance. It checks whether sufficient money is available before deducting a fare. It prevents the balance from falling below Rs 50.

Passenger: Stores passenger information. The category determines whether the passenger is eligible for a concession.

Journey: Represents one metro trip. It stores the destination zone and journey time. The time is used to determine whether peak-hour surcharge applies.

Zone: Represents the different metro zones. An enum class can be used to define the valid zones.

FareTable: Stores the fare values and calculation rules in one place. This prevents fare values from being repeated throughout the program.

1.2 Procedural programming versus OOP

In procedural programming, the system uses global variables and separate functions to calculate fares and update balances.

For example:

```
long long balance = 10000;


long long calculateFare(int zone);
void deductFare(long long fare);
bool canTravel(long long fare);
```

In object-oriented programming, the data and functions are combined into classes.

For example:

```
class SmartCard {
private:
   long long balance;


public:
   bool canTravel(long long fare);
   bool deduct(long long fare);
};
```

The object-oriented approach is safer because the balance is private and can only be modified through authorized methods.

1.3 Basic principles of OOP

Principle	**Application**
Encapsulation	SmartCard hides the balance and allows changes only through methods.

Abstraction	The gate calls calculateFare() without knowing the internal calculation.
Inheritance	StudentPassenger and SeniorPassenger can inherit common passenger information from Passenger.
Polymorphism	Different discount rules can implement a common fare-calculation interface.

1.4 Rules every correct journey must satisfy

1. The card balance must remain at least Rs 50 after fare deduction.
2. The fare must depend on the farthest zone travelled.
3. A 20% surcharge must apply only during peak hours.
4. Students and senior citizens must receive the approved concession.
5. All monetary calculations must use integer paise.
6. Recharge amounts must be positive.
7. Invalid zones and times must be rejected.
8. Fare values must be maintained in one location.

1.5 Alternative Concept and Complexity Analysis

If the system is implemented using procedural programming, global variables store the card balance and fare values. Separate functions perform fare calculations and balance deductions.

Both procedural and object-oriented approaches require a fixed number of operations for each journey.

- Time complexity per journey: O(1).
- Space complexity per card: O(1).
- Memory for a long long balance: typically 8 bytes.

The main difference is maintainability. In procedural programming, every function accessing the balance must be checked when a rule changes. In OOP, the SmartCard class controls the balance and enforces the rules in one place.

Conclusion: The object-oriented approach is more suitable because it provides better security, reusability, maintainability and data protection.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly

2.1 Five types of C++ tokens

Tokens are the smallest individual elements of a C++ program.

Token Type	Examples	Description
Keywords	int, class, if, else, return	Reserved words in C++.
Identifiers	SmartCard, balance, fare	Names given by programmers.
Constants	1000, 2000, 5000, true	Fixed values used in programs.
Operators	+, -, *, /, %, ==	Perform operations.
Punctuators	, ( ), ;, :	Separate and organize statements.

2.2 Difference between #define, const and constexpr

Feature	**#define**	**const**	**constexpr**
Processing	Preprocessor	Compiler	Compiler
Type checking	No	Yes	Yes
Scope rules	Not normal C++ scope	Yes	Yes
Compile-time evaluation	Text substitution	Depends on initialization	Guaranteed for constant expressions
Recommended use	Limited	Read-only values	Compile-time constants

Example:

```
constexpr long long INNER_FARE = 1000;
constexpr long long MIDDLE_FARE = 2000;
constexpr long long OUTER_FARE = 3000;

constexpr int PEAK_SURCHARGE = 20;
constexpr int CONCESSION = 50;

constexpr long long MIN_BALANCE = 5000;
```

constexpr is preferred because it provides type checking, compile-time validation and better scope management.

2.3 Difference between enum and enum class

Traditional enum:

```
enum Zone {
   Inner,
   Middle,
   Outer
};
```

Enum class:

```
enum class Zone {
   Inner,
   Middle,
   Outer
};
```

The enum class is safer because its values are scoped and do not automatically convert to integers.

For example:

```
Zone z = Zone::Inner;
```

This avoids accidental mixing of zone values with ordinary integer variables.

2.4 Naming rules and conventions

- Identifiers must begin with a letter or underscore.
- They cannot begin with a number.
- Keywords cannot be used as identifiers.
- C++ identifiers are case-sensitive.
- Constants should use uppercase letters with underscores.
- Class names should use PascalCase.

- Variables and functions should use camelCase.

Examples:

class SmartCard;

long long balance;

void calculateFare();

constexpr int PEAK_SURCHARGE = 20;

2.5 Alternative Concept and Complexity Analysis

If macros were used instead of constexpr, the compiler would replace the macro name with its defined text before compilation.

Macros do not provide normal type checking or scope protection.

Both macros and constexpr generally require no additional storage for simple compile-time constants. A fare array requires O(z) memory for z zones.

Both approaches have O(1) execution time for a simple fare lookup.

Conclusion: constexpr and enum class are safer and more maintainable than macros and traditional enum declarations.

TASK 3 – Choose Data Types and Check Type Compatibility

3.1 Selection of suitable data types

Data	**Data Type**	**Reason**

Fare	long long	Stores monetary values in paise.
Balance	long long	Supports large balance values.
Hour	int	Stores hours from 0 to 23.
Minute	int	Stores minutes from 0 to 59.
Zone	enum class	Represents valid metro zones.
Journey count	long long	Supports large numbers of journeys.
Peak status	bool	Stores true or false.
Passenger name	string	Stores text information.
Temporary value	auto	Automatically determines the type.

3.2 Why should money be stored in integer paise?

Floating-point numbers cannot represent every decimal value exactly.

For example:

#include <iostream>

```
using namespace std;

int main() {
   double a = 0.1;
   double b = 0.2;

   cout << a + b;
}
```

Possible output:

0.30000000000000004

This happens because floating-point values are stored using binary representation.

In a fare system, even a small rounding error can produce incorrect financial calculations.

Therefore, money should be stored in integer paise.

Example:

- Rs 10.00 = 1000 paise.
- Rs 20.00 = 2000 paise.
- Rs 30.00 = 3000 paise.
- Rs 50.00 = 5000 paise.

This guarantees exact arithmetic for these amounts.

3.3 Type compatibility

Type compatibility refers to how different data types work together in expressions.

Examples:

Integer arithmetic

```
int a = 7;
int b = 2;

cout << a / b;
```

Output:

3

Integer division removes the fractional part.

Integer and double conversion

```
int a = 7;
double b = 2.0;

cout << a / b;
```

Output:

3.5

Here, the integer is converted into a double.

Boolean conversion

```
bool value = 5;
cout << value;
```

Output:

1

Any non-zero value converts to true.

3.4 Range of data types

On a typical 64-bit system:

Data Type	Typical Size
bool	1 byte
char	1 byte
int	4 bytes
long long	8 bytes
float	4 bytes
double	8 bytes

The typical maximum value of a 32-bit signed int is:

2,147,483,647

The maximum value of a signed long long is approximately:

9.22 × 10^18

The whole-network daily collection can exceed the int limit. Therefore, long long is recommended for total fare collection.

3.5 Alternative Concept: Double instead of long long

If money is stored using double, calculations become easier to write, but rounding errors may occur.

Feature	**long long paise**	**double rupees**
Precision	Exact integer arithmetic	Approximate decimal representation
Typical size	8 bytes	8 bytes
Addition time	O(1)	O(1)
Financial accuracy	High	Rounding errors possible

Conclusion: Integer paise is the best choice for the metro fare system because it provides exact monetary calculations and avoids floating-point errors.

TASK 4 – Control Implicit Conversions

4.1 Implicit type conversion

Implicit conversion occurs when the compiler automatically converts one data type into another without requiring an explicit instruction from the programmer.

Types of implicit conversions

1. Integer division

```
int a = 7;
int b = 2;
```

```
double c = a / b;
cout << c;
```

Output:

3

The division is performed using integers first. Therefore, the fractional part is lost before assigning the result to double.

2. Integer-to-double conversion

```
int a = 7;
int b = 2;

double result = static_cast<double>(a) / b;
cout << result;
```

Output:

3.5

The static_cast converts the integer into double before division.

3. Character conversion

```
char ch = 'A' + 2;

cout << ch << " " << ch + 1;
```

Output:

C 68

The character is promoted to an integer during arithmetic.

4. Boolean conversion

```
bool flag = 5;

cout << flag << " " << boolalpha << flag;
```

Output:

```
1 true
```

Any non-zero integer is converted to true.

5. Signed and unsigned conversion

```
unsigned int u = 1;
int s = -1;

cout << (s < u ? "s < u" : "s >= u");
```

Output on a typical system:

```
s >= u
```

The signed integer is converted to unsigned during comparison.

4.2 Narrowing conversions

Narrowing occurs when a value is converted into a type that cannot represent the original value without losing information.

Example:

```
int x = 3.7;
```

The fractional part is lost, so x becomes 3.

Brace initialization prevents this type of narrowing:

```
int x{3.7};  // Compilation error
```

4.3 Use of static_cast

The static_cast operator performs an explicit type conversion.

Example:

```
int a = 10;
double b = static_cast<double>(a);


cout << b;
```

Output:

10

It makes conversions clear and helps avoid unintended operations.

4.4 Alternative Concept and Complexity Analysis

If all conversions were explicitly written using static_cast, the program would become easier to understand and maintain.

Both implicit and explicit conversions generally have O(1) time complexity and require no additional significant memory.

Explicit conversions improve readability and reduce unexpected data loss, while brace initialization helps detect narrowing errors during compilation.

Conclusion: Explicit conversions should be used whenever they improve clarity or prevent errors in financial calculations.

TASK 5 – Write Correct Expressions with the Right Precedence

5.1 Operator precedence

Operator precedence determines the order in which operators are evaluated in an expression.

For example:

```
int a = 5;
int b = 3;


cout << a + b * 2 << endl;
cout << (a + b) * 2;
```

Output:

```
11
16
```

In the first expression, multiplication is performed before addition.

In the second expression, parentheses give priority to addition.

5.2 Arithmetic operators

Operator	**Meaning**	**Example**
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b

%	Modulus	a % b

Example:

```
int a = 10;
int b = 3;


cout << a / b << endl;
cout << a % b;
```

Output:

```
3
1
```

5.3 Increment and decrement operators

There are two types of increment operators.

Post-increment: Returns the original value and then increments it.

Pre-increment: Increments the value first and then returns the updated value.

Example:

```
int x = 5;
int y = 3;


int z = x++ + ++y;


cout << z << endl;
cout << x << " " << y;
```

Output:

9

6 4

5.4 Conditional operator

The conditional operator is used to select one of two values.

Syntax:

condition ? expression1 : expression2;

Example:

```
int a = 10;
int b = 20;

int maximum = (a > b) ? a : b;

cout << maximum;
```

Output:

20

5.5 Expressions used in the metro fare system

The peak-hour condition can be written as:

```
bool peak =
   (time >= 480 && time < 600) ||
   (time >= 1020 && time < 1200);
```

Here, time represents minutes after midnight.

The surcharge calculation can be written as:

fare = fare + (fare * 20 / 100);

Parentheses make the calculation easier to understand.

5.6 Alternative Concept and Complexity Analysis

A peak-hour lookup table can be used instead of writing multiple logical conditions.

For example, a table containing 24 Boolean values can indicate whether each hour is peak or off-peak.

Both methods have O(1) time complexity.

The lookup table requires additional O(24) memory, while the expression requires no additional table storage.

However, a lookup table must be updated whenever the peak-hour rules change.

Conclusion: Correct operator precedence and parentheses are important for producing accurate fare calculations.

TASK 6 – Design the Control Structures

6.1 Selection of control structures

Control structures determine the order in which statements are executed.

The metro fare system uses if-else, switch, conditional operators and loops.

Control Structure	**Application**	**Reason**
if-else	Peak-hour checking	Evaluates conditions.

switch	Zone-based fare calculation	Selects fare based on zone.
Conditional operator	Allow or deny messages	Useful for simple two-way choices.
for loop	Processing a fixed number of journeys	Repeats a known number of times.
while loop	Reading journeys until input ends	Suitable for unknown input counts.
Range-based for	Processing journey records	Visits each record without requiring an index.

6.2 Use of break and continue

break: Terminates a loop or prevents execution from continuing into another switch case.

continue: Skips the current iteration and moves to the next iteration.

Example:

```
for (int i = 1; i <= 5; i++) {
   if (i == 3)
      continue;

   cout << i << " ";
}
```

Output:

1 2 4 5

6.3 What happens when no switch case matches?

If no case matches, none of the case statements executes unless a default case is provided.

Therefore, a default case should reject invalid zone values.

6.4 Algorithm for calculating metro fare

Algorithm: CalculateFare

1. Start.
2. Read passenger zone, time and passenger category.
3. Select the base fare using the zone.
4. Check whether the passenger is eligible for concession.
5. Apply the concession if eligible.
6. Check whether the journey occurs during peak hours.
7. Apply the peak surcharge if required.
8. Return the calculated fare.
9. Stop.

Algorithm: ProcessJourney

1. Start.
2. Read the card balance.
3. Calculate the journey fare.
4. Check whether the remaining balance is at least Rs 50.
5. If the condition is true, deduct the fare.
6. Otherwise, reject the journey.
7. Display the updated balance.
8. Stop.

6.5 Completed C++ program

```
#include <iostream>
#include <iomanip>
using namespace std;

enum class Zone {
   Inner, Middle, Outer
};

constexpr int BASE_PAISE[] = {
   1000, 2000, 3000
};

constexpr int PEAK_SURCHARGE_PERCENT = 20;

constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak) {

   long long base = 0;

   switch (z) {

      case Zone::Inner:
         base = BASE_PAISE[0];
         break;

      case Zone::Middle:
         base = BASE_PAISE[1];
         break;
```

```
        case Zone::Outer:
            base = BASE_PAISE[2];
            break;

        default:
            return -1;
    }

    if (peak) {
        base += base * PEAK_SURCHARGE_PERCENT / 100;
    }

    return base;
}

bool canTravel(long long balance, long long f) {

    return f >= 0 &&
           balance - f >= MIN_BALANCE_PAISE;
}

int main() {

    long long balance = 10000;

    struct Trip {
        Zone zone;
        bool peak;
    };

    Trip trips[] = {
```

```
        {Zone::Inner, false},
        {Zone::Outer, true},
        {Zone::Middle, false}
    };

    for (const Trip& t : trips) {

        long long f = fare(t.zone, t.peak);

        if (canTravel(balance, f)) {

            balance -= f;

            cout << "Fare Rs "
                 << f / 100 << "."
                 << setw(2) << setfill('0') << f % 100
                 << setfill(' ');

        } else {

            cout << "Denied (low balance)";
        }

        cout << " balance Rs "
             << balance / 100 << endl;
    }

    return 0;
}
```

Expected Output:

Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54

6.6 Alternative Concept and Complexity Analysis

Instead of using a switch statement, an array can be used to store fare values.

```
constexpr long long FARE[] = {
    1000, 2000, 3000
};
```

Both switch and array-based methods provide O(1) fare lookup.

An array uses O(z) memory for z zones. A switch may be implemented using a jump table by the compiler.

An array makes adding new zones easier, but the index must be validated to avoid accessing memory outside the array.

Conclusion: Switch statements are suitable for a small fixed number of zones, while arrays provide a convenient solution when the number of zones increases.

TASK 7 – From Procedural to Object-Oriented Programming

7.1 Procedural programming approach

In procedural programming, data and functions are maintained separately.

Example:

```
#include <iostream>
using namespace std;

long long balance = 10000;
```

```
const long long MIN_BALANCE = 5000;

bool canTravel(long long fare) {

   return balance - fare >= MIN_BALANCE;
}

void deduct(long long fare) {

   balance -= fare;
}

int main() {

   long long fare = 1000;

   if (canTravel(fare)) {

      deduct(fare);

      cout << "Payment successful" << endl;
      cout << "Balance: " << balance;
   }

   else {

      cout << "Journey denied";
   }

   return 0;
```

```
}
```

The major disadvantage is that the balance is stored globally and any function can modify it.

7.2 Object-oriented programming approach

In OOP, data and related functions are combined into a class.

Example:

```
#include <iostream>
using namespace std;

class SmartCard {

private:

    long long balance;

    static constexpr long long MIN_BALANCE = 5000;

public:

    SmartCard(long long b) {

        balance = b;
    }

    long long getBalance() const {

        return balance;
    }
```

```
    bool canTravel(long long fare) const {

        return fare >= 0 &&
               balance - fare >= MIN_BALANCE;
    }

    bool deduct(long long fare) {

        if (!canTravel(fare)) {

            return false;
        }

        balance -= fare;

        return true;
    }

    bool recharge(long long amount) {

        if (amount <= 0) {

            return false;
        }

        balance += amount;

        return true;
    }
};
```

```
int main() {

   SmartCard card(10000);

   if (card.deduct(1000)) {

      cout << "Payment successful" << endl;
   }

   else {

      cout << "Payment denied" << endl;
   }

   cout << "Balance: " << card.getBalance();

   return 0;
}
```

7.3 How does the class prevent a negative balance?

The SmartCard class declares balance as private.

The balance can only be modified using the available public methods.

Before deducting the fare, the deduct() function checks whether the remaining balance is at least Rs 50.

If the condition is false, the transaction is rejected.

Therefore, unauthorized balance modifications are prevented.

7.4 Comparison

Feature	Procedural	Object-oriented
Data storage	Global variables	Private class members
Security	Less secure	Better data protection
Reusability	Limited	High
Maintenance	Difficult for large programs	Easier
Time complexity	O(1)	O(1)
Space complexity	O(1)	O(1)

7.5 Alternative Concept: Public balance

If the balance is declared public, any function can directly modify it.

Example:

```
struct SmartCard {

   long long balance;
};

int main() {
```

```
    SmartCard card{10000};

    card.balance = -500;

    cout << card.balance;

    return 0;
}
```

Output:

-500

This violates the minimum-balance rule.

The time and space complexity remain O(1), but the program becomes less secure.

Conclusion: The object-oriented approach is preferable because encapsulation protects the card balance and prevents unauthorized modifications.

TASK 8 – Read and Print Data Safely

8.1 Reading data using cin

In C++, the cin object is used to read input from the keyboard.

Example:

```
int age;

cout << "Enter age: ";
cin >> age;

cout << "Age: " << age;
```

If the user enters an invalid value such as abc instead of a number, cin enters a failure state.

To recover from this situation, the following functions are used:

- cin.fail() – Checks whether input failed.
- cin.clear() – Clears the failure state.
- cin.ignore() – Removes invalid input from the buffer.

8.2 Safe input validation program

```
#include <iostream>
#include <iomanip>
#include <limits>
using namespace std;

int main() {

    int n;

    cout << "Enter number of journeys: ";

    while (!(cin >> n) || n < 0) {

        cout << "Invalid input. Enter again: ";

        cin.clear();

        cin.ignore(
            numeric_limits<streamsize>::max(),
            '\n'
        );
    }
```

```
    cout << "\nNumber of journeys: " << n;

    return 0;
}
```

8.3 Formatting output using iomanip

The iomanip library provides functions for formatting output.

Function	**Purpose**
setw()	Sets the width of output.
setfill()	Fills empty spaces with a character.
fixed	Displays floating-point values in fixed notation.
setprecision()	Controls decimal precision.

Example:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {

    double fare = 10.5;
```

```
    cout << fixed << setprecision(2);

    cout << "Fare: Rs " << fare;

    return 0;
}
```

Output:

Fare: Rs 10.50

8.4 Metro journey report

The program should display a structured report containing:

1. Journey number.
2. Destination zone.
3. Journey time.
4. Fare amount.
5. Transaction status.
6. Remaining balance.

Example output:

```
----------------------------------------
      METRO JOURNEY REPORT
----------------------------------------
Journey No     : 1
Zone           : Inner
Journey Time   : 08:30
Fare           : Rs 12.00
Status         : Approved
Balance        : Rs 88.00
```

8.5 Alternative Concept: getline() and stoi()

Instead of reading values directly using cin, the program can use getline() to read an entire line and stoi() to convert the text into an integer.

Example:

```
#include <iostream>
#include <string>
using namespace std;

int main() {

    string input;

    cout << "Enter journey count: ";

    getline(cin, input);

    try {

        size_t pos;

        int n = stoi(input, &pos);

        if (pos != input.size() || n < 0) {
            throw invalid_argument("Invalid input");
        }

        cout << "Journeys: " << n;
    }
```

```
    catch (...) {

        cout << "Invalid input";
    }

    return 0;
}
```

Both cin and getline process input in O(L) time for a line of length L.

getline() requires a buffer proportional to the input length.

Conclusion: Input validation is important because invalid data can cause incorrect fare calculations and unexpected program behaviour.

TASK 9 – Implement and Test the Fare Logic (4 Marks)

9.1 Implementation of fare calculation

The fare calculation must consider:

- Destination zone.
- Peak or off-peak time.
- Passenger concession.
- Minimum balance requirement.

Money is stored in paise to maintain accuracy.

C++ program

```
#include <iostream>
using namespace std;

enum class Zone {
```

```
    Inner, Middle, Outer
};

constexpr long long MIN_BALANCE = 5000;

long long calculateFare(
    Zone zone,
    bool peak,
    bool concession
) {

    long long fare = 0;

    switch (zone) {

        case Zone::Inner:
            fare = 1000;
            break;

        case Zone::Middle:
            fare = 2000;
            break;

        case Zone::Outer:
            fare = 3000;
            break;

        default:
            return -1;
    }
```

```
    if (concession) {

        fare = (fare * 50 + 50) / 100;
    }

    if (peak) {

        fare = (fare * 120 + 50) / 100;
    }

    return fare;
}

bool canTravel(long long balance, long long fare) {

    return fare >= 0 &&
           balance - fare >= MIN_BALANCE;
}

int main() {

    long long balance = 10000;

    long long fare = calculateFare(
        Zone::Inner,
        true,
        false
    );

    if (canTravel(balance, fare)) {
```

```
        balance -= fare;

        cout << "Journey Approved" << endl;
        cout << "Fare: Rs " << fare / 100.0 << endl;
        cout << "Balance: Rs " << balance / 100.0;
    }

    else {

        cout << "Journey Denied";
    }

    return 0;
}
```

9.2 Test cases

Test Case	**Input**	**Expected Result**
TC01	Inner, off-peak, regular	Rs 10
TC02	Middle, peak, regular	Rs 24
TC03	Outer, off-peak, concession	Rs 15
TC04	Inner at 08:00	Peak surcharge applies

TC05	Inner at 10:00	No peak surcharge
TC06	Inner at 17:00	Peak surcharge applies
TC07	Inner at 20:00	No peak surcharge
TC08	Invalid zone	Reject journey
TC09	Negative recharge	Reject recharge
TC10	Remaining balance exactly Rs 50	Allow journey

9.3 Testing and validation

Testing is performed to verify that the program produces correct results for different inputs.

The important test categories are:

Normal testing: Checks whether valid journeys produce the correct fare.

Boundary testing: Checks exact peak-hour boundaries such as 08:00, 10:00, 17:00 and 20:00.

Invalid input testing: Checks whether incorrect zones, negative values and invalid times are rejected.

Balance testing: Checks whether the remaining balance stays at least Rs 50.

9.4 Alternative Concept: Table-driven testing

Instead of writing separate test statements, all test cases can be stored in an array or vector.

A loop can execute every test case and compare the actual result with the expected result.

For t test cases:

- Time complexity: O(t).
- Space complexity: O(t).

Conclusion: Testing and validation ensure that the metro fare system produces accurate results and handles invalid situations safely.

TASK 10 – Compare Expectations with Execution

10.1 Importance of comparing expected and actual results

A program must be tested by comparing its expected output with its actual output.

This helps identify:

- Incorrect calculations.
- Integer overflow.
- Floating-point errors.
- Invalid input handling.
- Incorrect peak-hour conditions.
- Balance deduction errors.

10.2 Daily journey processing

The metro network may grow from one station to the entire network.

Network Level	Journeys per Day

One station	20,000
One line	500,000
Whole network	3,000,000

The system must use suitable data types to process these large numbers.

10.3 Comparison of int and long long

The typical maximum value of a signed int is:

2,147,483,647

The maximum value of long long is approximately:

9,223,372,036,854,775,807

The reference solution gives an example whole-network collection of 5,626,462,900 paise.

This value is greater than the maximum value of int.

Therefore, int is not suitable for storing the total collection at whole-network scale.

Comparison table

Data Type	**Typical Size**	**Suitability**
int	4 bytes	Limited range

long long	8 bytes	Suitable for large totals
float	4 bytes	Insufficient precision for large financial totals
double	8 bytes	Approximate floating-point calculations

10.4 Complexity analysis

If the system processes n journeys:

- Time complexity for processing all journeys: O(n).
- Time complexity per journey: O(1).
- Space complexity for one accumulator: O(1).

Using long long prevents the integer-range problem for the scenario's totals.

10.5 Compiler warnings

Compiler warnings help identify potential problems.

Important compiler options:

g++ -std=c++17 -Wall -Wextra program.cpp -o program

Some useful warnings include:

- -Wsign-compare
- -Wswitch
- -Wconversion

These help identify incorrect signed/unsigned comparisons, incomplete switch statements and potentially unsafe conversions.

10.6 Alternative Concept and Complexity Analysis

Using int requires less memory than long long, but its limited range creates overflow risks.

Using double allows decimal calculations but introduces precision issues.

Using long long for integer paise provides exact calculations and sufficient range for the given metro system.

Conclusion: Long long is the recommended data type for fare collection and balance calculations because it supports large values without floating-point rounding errors.

TASK 11 – Recommend and Defend the Final Design

11.1 Recommended design

After analysing the requirements, an object-oriented design using SmartCard, Passenger, Journey and FareTable classes is recommended.

Recommended data types and classes

Component	Recommendation	Reason
Fare	long long	Exact paise calculations
Balance	long long	Supports large values
Zone	enum class	Restricts valid zone values

Constants	constexpr	Centralized fare definitions
SmartCard	Class	Protects balance
Passenger	Class	Maintains passenger information
Journey	Class	Stores journey details
FareTable	Class	Maintains fare calculation rules

11.2 How will a fare revision be carried out?

If the metro changes the fare of a zone, only the corresponding fare constant needs to be updated.

Example:

```
constexpr long long INNER_FARE = 1000;
constexpr long long MIDDLE_FARE = 2000;
constexpr long long OUTER_FARE = 3000;
```

If the Inner zone fare changes from Rs 10 to Rs 12, update:

```
constexpr long long INNER_FARE = 1200;
```

After changing the fare, the program must be compiled and tested again.

11.3 Limitations of the design

1. The current fare system supports only three zones.
2. Fare values are compiled into the program.

3. New discount rules may require additional changes.
4. Monthly passes are not included in the original design.
5. The concession percentage is based on an assumption and requires confirmation.
6. A production system would also require secure transaction management.

11.4 Alternative Concept: Configuration file

Instead of storing fare values directly in the program, the system can read them from a configuration file when it starts.

Example configuration:

INNER=1000
MIDDLE=2000
OUTER=3000
PEAK_SURCHARGE=20
MIN_BALANCE=5000

The program reads these values during startup.

Comparison

Feature	**constexpr**	**Configuration File**
Fare revision	Requires recompilation	File can be updated
Startup time	No file loading	Requires reading the file
Memory	Compile-time constants	Stores loaded fare values

Flexibility	Less flexible	More flexible
Validation	Compiler checks constants	Program must validate file data

Reading z fare values requires O(z) time and O(z) storage.

Conclusion: The recommended design uses integer paise, constexpr constants, enum class and private SmartCard balance. It is accurate, secure, maintainable and suitable for processing the whole metro network.

TASK 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge)

Changed Requirements

The metro system introduces three new requirements:

1. A fourth zone called Airport with a fare of Rs 60.
2. A 10% weekend discount that does not combine with the peak surcharge.
3. A monthly pass that allows unlimited journeys in two selected zones.

The original design must be evaluated and modified to support these requirements.

A. Impact Analysis

The new requirements affect several components of the original program.

Original Component	**Required Modification**
Zone enum	Add Airport zone.

Fare table	Add Airport fare of Rs 60.
Fare constants	Add weekend discount percentage of 10%.
Peak-hour condition	Prevent peak surcharge from combining with weekend discount.
Journey class	Include journey date and weekend information.
SmartCard class	Maintain the existing minimum-balance rule.
MonthlyPass class	Introduce a new class for monthly passes.

Explanation

The original system supports only three zones. Adding the Airport zone requires modifying the zone enumeration and fare table.

The weekend discount introduces another fare calculation rule. Therefore, the program must identify whether a journey occurs on a weekend.

The monthly pass requires a separate class that stores the validity month and the two selected zones.

B. Evaluation of the Original Design

The original object-oriented design is useful, but some modifications are necessary.

1. Switch-based fare logic

The original switch statement supports three zones.

When the Airport zone is introduced, a new case must be added.

```
case Zone::Airport:
    fare = 6000;
    break;
```

As the number of zones increases, maintaining a large switch statement becomes difficult.

Therefore, an array-based fare table is more suitable.

2. SmartCard class

The SmartCard class can still be retained because it protects the card balance and ensures that at least Rs 50 remains after payment.

However, monthly pass users should not have their stored balance deducted for journeys covered by the pass.

3. Worked examples

Journey	**Calculation**	**Final Fare**
Airport, weekday off-peak	Rs 60	Rs 60
Airport, weekday peak	Rs 60 + 20%	Rs 72
Airport, weekend	Rs 60 − 10%	Rs 54

Airport, weekend peak time	Weekend discount only	Rs 54
Inner zone with valid monthly pass	No fare deduction	Rs 0

The weekend discount and peak surcharge must never be combined.

Evaluation: The original design can support the new requirements after modifying the fare table and introducing separate discount and monthly-pass components.

C. Adaptation or Redesign

The system should be modified rather than completely redesigned.

The following components are introduced:

- Updated Zone enumeration.
- Fare table using an array.
- Weekend discount calculation.
- MonthlyPass class.
- Updated SmartCard class.

Updated C++ Program

```
#include <iostream>
#include <array>
#include <optional>
using namespace std;

enum class Zone : int {
    Inner,
    Middle,
    Outer,
```

```
    Airport,
    Count
};

constexpr array<long long, 4> BASE_PAISE = {
    1000, 2000, 3000, 6000
};

constexpr int PEAK_PERCENT = 20;

constexpr int WEEKEND_DISCOUNT = 10;

constexpr long long MIN_BALANCE = 5000;
bool isPeak(int hour, int minute) {

    if (hour < 0 || hour > 23 ||
        minute < 0 || minute > 59) {
        return false;
    }

    int time = hour * 60 + minute;

    return (time >= 480 && time < 600) ||
           (time >= 1020 && time < 1200);
}
long long percentOf(long long amount, int percent) {

    return (amount * percent + 50) / 100;
}
optional<long long> calculateFare(
    Zone zone,
```

```
    bool peak,
    bool weekend
) {

    int index = static_cast<int>(zone);

    if (index < 0 ||
        index >= static_cast<int>(Zone::Count)) {

        return nullopt;
    }

    long long fare = BASE_PAISE[index];

    if (weekend) {

        fare -= percentOf(
            fare,
            WEEKEND_DISCOUNT
        );

    }

    else if (peak) {

        fare += percentOf(
            fare,
            PEAK_PERCENT
        );
    }
```

```
    return fare;
}
class MonthlyPass {

private:

    int year;
    int month;

    Zone firstZone;
    Zone secondZone;

public:

    MonthlyPass(
        int y,
        int m,
        Zone a,
        Zone b
    ) {

        year = y;
        month = m;

        firstZone = a;
        secondZone = b;
    }

    bool covers(
        Zone zone,
        int y,
```

```
        int m
    ) const {

        return y == year &&
               m == month &&
               (zone == firstZone ||
                zone == secondZone);
    }
};
class SmartCard {

private:

    long long balance;

public:

    SmartCard(long long b) {

        balance = b;
    }

    long long getBalance() const {

        return balance;
    }

    bool deduct(long long fare) {

        if (fare < 0 ||
            balance - fare < MIN_BALANCE) {
```

```
            return false;
        }

        balance -= fare;

        return true;
    }
};


int main() {

    static_assert(
        BASE_PAISE.size() ==
        static_cast<size_t>(Zone::Count)
    );

    SmartCard card(20000);

    MonthlyPass pass(
        2026,
        11,
        Zone::Inner,
        Zone::Middle
    );

    Zone tripZone = Zone::Airport;

    bool weekend = true;
```

```
bool peak = isPeak(18, 0);

if (pass.covers(tripZone, 2026, 11)) {

    cout << "Monthly Pass Accepted";
}

else {

    auto fare = calculateFare(
        tripZone,
        peak,
        weekend
    );

    if (!fare) {

        cout << "Invalid Zone";
    }

    else if (card.deduct(*fare)) {

        cout << "Journey Approved" << endl;

        cout << "Fare: Rs "
             << *fare / 100.0 << endl;

        cout << "Balance: Rs "
             << card.getBalance() / 100.0;
    }
```

```
        else {

            cout << "Journey Denied";
        }
    }

    return 0;
}
```

Expected Output

Journey Approved
Fare: Rs 54
Balance: Rs 146

The Airport zone has a base fare of Rs 60. The weekend discount is 10%, resulting in Rs 54. The peak surcharge is not applied because the weekend discount takes priority.

The monthly pass allows unlimited journeys in the two selected zones during its valid month.

D. Evidence-Based Defence

The modified design is better suited to the new metro requirements.

Advantages

1. Scalability: The fare array makes it easier to add additional zones.
2. Accuracy: Integer paise avoids floating-point calculation errors.
3. Flexibility: Weekend discount and peak surcharge are maintained as separate rules.
4. Reusability: The MonthlyPass class can be reused for different passengers.
5. Security: SmartCard protects the balance using private data members.
6. Maintainability: Changes in fare rules can be made without rewriting the entire program.

Complexity Analysis

Operation	Time Complexity	Space Complexity
Fare lookup	O(1)	O(1) per lookup
Peak-hour checking	O(1)	O(1)
Weekend discount	O(1)	O(1)
Monthly pass validation	O(1)	O(1)
SmartCard deduction	O(1)	O(1)

Final Decision: MODIFY

The original object-oriented design should be modified rather than completely redesigned.

The SmartCard class, integer-paise calculations and encapsulation principles can be retained. The fare table should be extended to include the Airport zone, and separate weekend-discount and MonthlyPass components should be introduced.

This approach reduces unnecessary changes while improving flexibility, scalability and maintainability.

OVERALL CONCLUSION

The Namma Metro Smart-Card Fare System demonstrates how object-oriented programming concepts and C++ features can be used to develop a reliable and efficient fare-management application.

The use of suitable data types, constexpr constants, enum class, operators and control structures helps maintain accurate fare calculations. Integer paise avoids floating-point errors, while encapsulation protects the smart-card balance from unauthorized modifications.

Testing and validation ensure that the program handles peak hours, concessions, invalid inputs and minimum-balance conditions correctly.

The modified design supports the Airport zone, weekend discounts and monthly passes without replacing the entire original system.

Therefore, the recommended object-oriented design provides accuracy, security, scalability, flexibility and maintainability for the growing metro network.